

Branching

Mahir Labib Dihan
Lecturer, CSE, BUET



Table of Contents

1. Operators Continued
2. Introduction to Control Flow
3. The `if` Statement
4. The `if-else` Statement
5. Case Study: The Leap Year Problem
6. Nested `if` and `else-if` Ladders
7. The `switch` Statement
8. The Ternary Operator
9. Problem Solving

Operators Continued



Classification of Operators

Operators can be classified based on the number of operands they take:

1. **Unary Operators:** Act on a single operand.
 - Examples: ++ (Increment), -- (Decrement), - (Unary Minus), ! (Logical NOT), ~ (Bitwise NOT), sizeof.
2. **Binary Operators:** Act on two operands.
 - Examples: + (Add), - (Sub), * (Mul), / (Div), % (Mod), ==, &&, ||, &, |.
3. **Ternary Operator:** Acts on three operands.
 - Example: ? : (Conditional Operator).
 - *We will discuss this in detail later.*

Introduction to Control Flow



The Flow of Execution

Sequential Execution

By default, C programs execute statements one after another, from top to bottom.

However, real life is full of decisions:

- **If** it rains, take an umbrella.
- **If** the traffic light is red, stop; else, go.
- **Switch** the TV channel based on the button pressed.

To handle these, we need **Control Structures**.

Types of Control Structures

1. Sequential: Default mode.
2. Selection (Branching): Making decisions (Current Topic).
 - `if`
 - `if-else`
 - `else-if` ladder
 - `switch`
3. Iteration (Looping): Repeating tasks (Next Topic).
 - `for`, `while`, `do-while`

The i f Statement



The if Statement

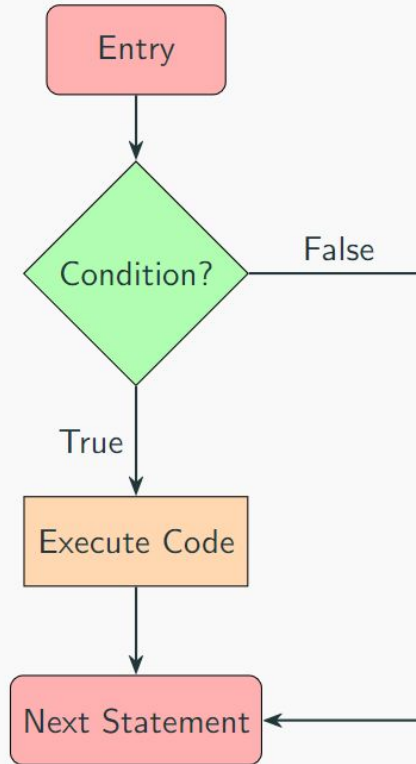
The simplest form of decision making.

Syntax

```
if (expression) {  
    // Statement(s) to execute if expression is TRUE  
}
```

- The expression is evaluated first.
- If it evaluates to **non-zero (True)**, the body is executed.
- If it evaluates to **zero (False)**, the body is skipped

Flowchart: if Statement



Example: Shopping Discount

Scenario: If the bill amount is over \$100, give a \$10 discount.

```
#include<stdio.h>
int main() {
    float bill = 120.50;
    if (bill > 100.0) {
        printf ("You get a discount!\n");
        bill = bill - 10.0;
    }
    printf("Final Bill: $%.2f\n", bill);
    return 0;
}
```

Output:

```
You get a discount!
Final Bill: $110.50
```

Scope and Braces

Important Rule

If the body of the if contains only one statement, the braces `{}` are optional but recommended.

Valid (Risky):

```
if (x > 0)
    printf("Positive");
```

Valid (Safe):

```
if (x > 0) {
    printf("Positive");
}
```

Common Mistake: The Semi-colon

What happens here?

```
int x = -5;  
if (x > 0); // <-- Note the semi-colon!  
{  
    printf("Number is positive.\n");  
}
```

Result: The `printf` executes regardless of the value of `x`.

- The `;` ends the `if` statement immediately.
- The block `{ . . . }` is treated as a separate, unconditional block of code

The if-else Statement



The if-else Statement

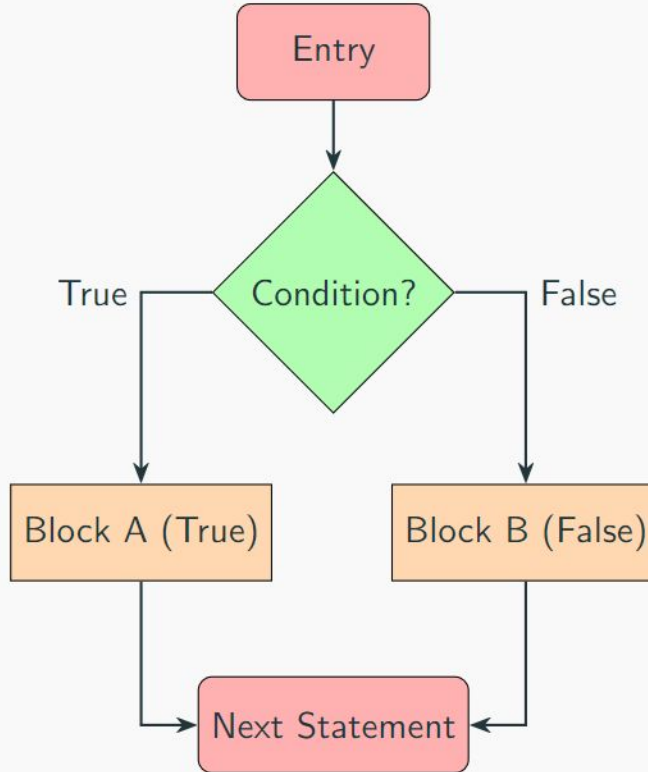
Used to choose between **two** alternatives.

Syntax

```
if (expression) {  
    // Block A (True)  
} else {  
    // Block B (False)  
}
```

One of the blocks is guaranteed to execute. Never both.

Flowchart: **if-else** Statement



Example: Odd or Even

```
#include<stdio.h>
int main() {
    int num;
    printf("Enter an integer: ");
    scanf("%d", &num);

    if ( num % 2 == 0) {
        printf("%d is Even.\n", num);
    } else {
        printf("%d is Odd.\n", num);
    }
    return 0;
}
```

Note: `num % 2` calculates the remainder.

Example: Password Checker

```
int stored_pin = 1234;
int input_pin;

printf("Enter PIN: ");
scanf("%d", &input_pin);

if (input_pin == stored_pin) {
    printf("Access Granted.\n");
    printf("Welcome User.\n");
} else {
    printf("Access Denied.\n");
    printf("Alarm Triggered!\n");
}
```

Structuring Logic: Vertical vs. Horizontal

When handling multiple conditions, we can structure our code in two main ways:

1. Vertical Extension

- Using nested if-else or else-if ladders.
- Logic flows downwards

Syntax

```
if (condition1) {  
    if (condition2) {  
        // if BOTH 1 and 2 are true  
    }  
}
```

2. Horizontal Extension

- Combining conditions in a single line using Logical Operators (&&, ||).
- Logic flows across.

Syntax

```
if (condition1 && condition2) {  
    // if BOTH 1 and 2 are true  
}
```

Case Study: The Leap Year Problem



Understanding the Logic

Problem: Determine if a given year is a Leap Year.

The Math Behind the Rules: (Solar Year ≈ 365.2422 days)

- **Every 4 years (+1 day):** Avg Year = 365.25 days.
Error: $365.25 - 365.2422 = 0.0078$ days/yr (Too high).
→ In 100 years, error accumulates to ≈ 0.78 days (≈ 1 day).
- **Every 100 years (Skip):** Remove that extra day. Avg = 365.24 days.
Error: $365.2422 - 365.24 = 0.0022$ days/yr (Too low).
→ In 400 years, error accumulates to ≈ 0.88 days (≈ 1 day).
- **Every 400 years (Add back):** Add the missing day. Avg = 365.2425.
→ Very close to 365.2422. Corrects the drift!

Examples:

- **2024:** Div by 4? Yes. Div by 100? No. → **Leap Year.**
- **1900:** Div by 4? Yes. Div by 100? Yes. Div by 400? No. → **Not Leap.**
- **2000:** Div by 4? Yes. Div by 100? Yes. Div by 400? Yes. → **Leap Year.**

Solution 1: Vertical Extension (Nested)

Checking conditions step-by-step.

```
if (year % 4 == 0) {  
    if (year % 100 == 0) {  
        if (year % 400 == 0) {  
            printf("Leap Year"); // Divisible by 400  
        } else {  
            printf("Common Year"); // Divisible by 100 but not 400  
        }  
    } else {  
        printf("Leap Year"); // Divisible by 4 but not 100  
    }  
} else {  
    printf("Common Year"); // Not divisible by 4  
}
```

Solution 2: Horizontal Extension

Combining conditions into a single expression.

Logic

(Divisible by 400) **OR** ((Divisible by 4) AND (Not Divisible by 100))

```
if ((year % 400 == 0) || ((year % 4 == 0) && (year % 100 != 0))) {  
    printf("Leap Year");  
} else {  
    printf("Common Year");  
}
```

Efficiency and Short-Circuiting

Which solution is better? It depends, but order matters!

Short-Circuit Evaluation

- `A || B`: If A is True, B is ignored.
- `A && B`: If A is False, B is ignored.

Optimization Tip: Put the most likely cases or “cheapest” checks first to take advantage of short-circuiting.

- Most years are NOT divisible by 4.
- Checking `year % 4 == 0` first allows us to exit early for 75% of cases!

Nest if and if-else Ladders



Nested if

Putting an if inside another if

Syntax

```
if (condition1) {  
    if (condition2) {  
        // Code executes if BOTH 1 and 2 are true  
    }  
}
```

Use this when the second decision depends on the first one passing

Example: Largest of Three Numbers

```
if (a > b) {  
    if (a > c) {  
        printf("A is largest");  
    } else {  
        printf("C is largest");  
    }  
} else {  
    // Here b >= a  
    if (b > c) {  
        printf("B is largest");  
    } else {  
        printf("C is largest");  
    }  
}
```

The Dangling else Problem

Which if does the else belong to?

Example

```
if (x > 0)
    if (y > 0)
        printf("Both Positive");
    else
        printf("???");
```

Rule: An else attaches to the nearest preceding if that doesn't have an else.

Solution: Always use braces {} to be explicit!

The else-if Ladder

Used when checking multiple mutually exclusive conditions.

Syntax

```
if (condition1) {  
    // Statement 1  
}  
else if (condition2) {  
    // Statement 2  
}  
else if (condition3) {  
    // Statement 3  
}  
else {  
    // Default Statement  
}
```

Example: Game Character Health

```
int health = 75;

if (health <= 0) {
    printf("Game Over\n");
}
else if (health < 20) {
    printf("Warning : Low Health!\n");
}
else if (health < 100) {
    printf("Player is active.\n");
}
else {
    printf("Full Health!\n");
}
```

Example: Coordinate Quadrants

Given (x, y), find the quadrant.

```
if (x > 0 && y > 0)
    printf("Quadrant I");
else if (x < 0 && y > 0)
    printf("Quadrant II");
else if (x < 0 && y < 0)
    printf("Quadrant III");
else if (x > 0 && y < 0)
    printf("Quadrant IV");
else
    printf("On Axis or Origin");
```

The Art of Indentation

Code is for Humans

“Any fool can write code that a computer can understand. Good programmers write code that humans can understand.” - Martin Fowler

Why Indent?

- **Readability:** Clearly shows logical structure.
- **Hierarchy:** Visualizes which statements belong to which block.
- **Debugging:** Helps spot missing braces } or logical errors, especially in nested logic.

Note: In C, indentation is ignored by the compiler, but it is critical for you (and whoever is reading your code)!

Good vs. Bad Indentation

Bad Code (Flat)

```
if(x>0){  
if(y>0){  
printf("Q1");  
}else{  
printf("Q4");  
}  
}else{  
printf("Left");  
}
```

Hard to trace.

Good Code (Indented)

```
if (x > 0) {  
    if (y > 0) {  
        printf("Q1");  
    } else {  
        printf("Q4");  
    }  
} else {  
    printf("Left");  
}
```

Structure is clear.

The switch Statement



The switch Statement

An alternative to the `else-if` ladder when testing a single variable against exact values.

Syntax

```
switch (variable) {  
    case value1:  
        // statements  
        break;  
    case value2:  
        // statements  
        break;  
    default:  
        // default statements  
}
```

switch Rules

1. The **expression** must evaluate to an **integer** or **character**. (No float, no string).
2. **Case values** must be constant literals (e.g., 5, 'A'). You cannot use variables like `case x:`.
3. **Break** prevents “fall-through”. Without it, execution continues to the next case.
4. **Default** is optional (like the final `else`).

Example: Simple Menu

```
int choice ;
printf("1. Start Game\n2. Load Game\n3. Exit\n");
scanf("%d", &choice) ;

switch (choice) {
    case 1:
        printf("Starting...\n");
        break;
    case 2:
        printf("Loading...\n");
        break;
    case 3:
        printf("Goodbye!\n");
        break;
    default:
        printf("Invalid input.\n");
}
```

Example: Vowel Checker (Fall-through)

Using fall-through feature to group cases.

```
char ch ;
scanf("%c", &ch);

switch (ch) {
    case 'a':
    case 'e':
    case 'i':
    case 'o':
    case 'u':
        printf("It is a vowel.\n");
        break;
    default:
        printf("It is a consonant (or special char).\n");
}
```

switch vs if-else

switch Case

- Works only with int / char.
- Checks equality only (==).
- Jump table implementation (often faster for many cases).
- Good for menus, state machines.

if-else Ladder

- Works with all types (float, etc.).
- Checks any condition (>, <, &&, etc.).
- Checks sequentially.
- Good for ranges and complex logic.

The Ternary Operator



Conditional Operator (? :)

A shorthand for if-else. It is the only operator in C that takes 3 operands.

Syntax

```
condition ? expression1 : expression2;
```

- If condition is **True**, result is expression1.
- If condition is **False**, result is expression2.

Example: Ternary vs If-Else

Task: Find the maximum of a and b.

Using if-else:

```
if (a > b)
    max = a;
else
    max = b;
```

Using Ternary:

```
max = (a > b) ? a : b;
```

Example: Absolute Value

Writing a concise absolute value macro or statement.

```
int num = -10;
int abs_val;

abs_val = (num < 0) ? -num : num;

printf("Absolute value: %d\n", abs_val);
// Output : 10
```

Nested Ternary (Use with Caution)

Checking for positive, negative, or zero.

```
int x = 5;

// Read as : Is x > 0? If yes , 1.
// Else , is x < 0? If yes , -1. Else 0.
int sign = (x > 0) ? 1 : ((x < 0) ? -1 : 0);
```

Note: While valid, nested ternary operators can be hard to read. Use `else-if` for clarity in complex logic.

Problem Solving



A Common Mistake

What is the output of the following code snippet?

```
int x = 10;  
if (x = 5) {  
    printf("True: %d", x);  
} else {  
    printf("False: %d", x);  
}
```

Output:

True: 5

Reason: The code uses = (assignment), not == (equality).

1. x is assigned 5.
2. The expression evaluates to 5 (non-zero), which is True.
3. The body executes.

Traffic Light System

Write a switch-case program that prints instructions based on character input:

- 'R' or 'r' → "Stop"
- 'Y' or 'y' → "Get Ready"
- 'G' or 'g' → "Go"
- Other → "Traffic Signal Error"

Traffic Light System Solution

```
char color ;
scanf (" %c" , &color); // Note space before %c

switch ( color ) {
    case 'R':
    case 'r':
        printf("Stop\n");
        break;
    case 'Y':
    case 'y':
        printf("Get Ready\n");
        break ;
    case 'G':
    case 'g':
        printf("Go\n");
        break;
    default:
        printf("Traffic Signal Error\n");
}
```

Problem: Real Roots of Quadratic Equation

Calculate roots for $ax^2 + bx + c = 0$.

Steps

1. Input coefficients a , b , and c .
2. Compute the discriminant $D = b^2 - 4ac$.
3. Use `if-else` to determine the nature of roots:
 - $D > 0$: Two distinct real roots.
 - $D == 0$: One real root (double root).
 - $D < 0$: No real roots (complex roots).

Solution Skeleton

```
float a, b, c, D, root1, root2;  
// ... input a, b, c ...  
D = b*b - 4*a*c;  
  
if (D > 0) {  
    root1 = (- b + sqrt(D)) / (2*a);  
    root2 = (- b - sqrt(D)) / (2*a);  
    printf("Roots: %.2f , %.2f", root1, root2);  
}  
else if (D == 0) {  
    root1 = -b / (2*a);  
    printf("Single Root: %.2f" , root1);  
}  
else {  
    printf("No Real Roots");  
}
```

Problem: Grading System

Problem: Write a program that converts marks (0-100) into letter grades.

Marks	Grade
90 - 100	A
80 - 89	B
70 - 79	C
60 - 69	D
0 - 59	F

Task: Solve this using:

1. if-else-if ladder.
2. switch-case statement.

Solution A: Grading with `if-else`

The natural choice for **ranges**.

```
int marks;
scanf("%d", &marks);

if (marks >= 90) {
    printf("Grade A");
} else if (marks >= 80) {
    printf("Grade B");
} else if (marks >= 70) {
    printf("Grade C");
} else if (marks >= 60) {
    printf("Grade D");
} else {
    printf("Grade F");
}
```

Solution B: Grading with switch

switch works on specific values, not ranges.

Trick: Divide marks by 10 to get the grouping prefix!

```
switch ( marks / 10) {  
    case 10:  
    case 9:  
        printf("Grade A"); break;  
    case 8:  
        printf("Grade B"); break;  
    case 7:  
        printf("Grade C"); break;  
    case 6:  
        printf("Grade D"); break;  
    default:  
        printf("Grade F");  
}
```

Note: Logic assumes $0 \leq \text{marks} \leq 100$.

Summary

1. Use `if` for single conditional execution.
2. Use `if-else` for “this or that” logic.
3. Use `else-if` ladders for ranges and complex multi-condition logic.
4. Use `switch` for checking discrete values (menus, commands).
5. Use `?:` (ternary operator) for simple assignments based on conditions.
6. Always verify indentation and braces to avoid logical errors.

Acknowledgement

- Mohammad Sadat Hossain, Lecturer, CSE, BUET